SIT313 – Secure Frontend Applications

Task P5: Newsletter Report

Student Name: Lav
Student ID: 2510999052

1. Introduction

The goal of this task is to link a React website with a Node.js and Express server for the DEV@Deakin online site. In this task I rebuilt the email newsletter registration part called "Sign Up To My Newsletter" on the page. When someone types their email and hits Subscribe the React website sends a web request to the Express server. The server checks the email sends a message through the SendGrid Email API and sends the right web status code back, to the website.

2. Technologies Used

- **React:** Used to create the frontend UI components, handle user state, and manage form submissions.
- **Node.js & Express.js:** Used to run the backend server, create API routes, and process incoming JSON requests.
- **SendGrid Mail API (@sendgrid/mail):** Used to send automated welcome emails to subscribed users.
- **CORS (cors):** Used to allow cross-origin requests between the React frontend (running on Vite) and the Express backend.
- **Dotenv (dotenv):** Used to securely read the SendGrid API key from the .env file.
- **CSS:** Used to style the Daily Insider newsletter section and display status feedback messages.

3. Implementation

3.1 Backend Server (server.js)

I set up an Express backend server that runs on port 3000.

1. CORS Middleware: I enabled CORS so that the React frontend can send POST requests to http://localhost:3000/subscribe without running into any browser security problems.

2. Data Validation: I added checks to confirm that the email exists and follows the email format. If the email is missing or invalid the server returns an HTTP 400 Request response.

3. SendGrid Integration: I securely loaded the SendGrid API key. I set up the email message, including the, from address to address, subject and body.

4. Error Handling & Status Codes: I used a block. When everything works the server logs a Status 200 message to the console. Sends an HTTP 200 OK response. If an error occurs the server sends an HTTP 500 Internal Server Error.

3.2 Frontend Sign-Up Component (signup.jsx)

I implemented the newsletter subscription component on the homepage:

1. **User Interface:** Added the heading **"SIGN UP FOR OUR DAILY INSIDER"**, an email input field, a Subscribe button, and a status message area.
2. **Client Validation:** Checks whether the email field is empty or contains an invalid format before sending the request.
3. **Asynchronous Request (fetch):** Sends a POST request with JSON data using async/await and try-catch.
4. **User Feedback:** The button displays "Subscribing..." while the request is processing. Upon receiving HTTP 200, it displays "Successfully subscribed! Welcome email sent." and clears the input field. If there is an error, it informs the user what went wrong.

4. Testing

I thoroughly tested the frontend and backend integration to make sure all requirements work as expected:

• **Empty Input Validation:** I clicked the Subscribe button without typing anything in the email field. The form stopped the submission immediately. Showed the message "Please enter your email address."

• **Invalid Email Format Validation:** I entered an email format without standard domain characters (e.g. lavsukhija). The frontend noticed the format and showed "Please enter a valid email address." Also the backend rejected the request with an HTTP 400 Request status code.

• **Successful Subscription Workflow:** I typed my email address (lavsukhija001@gmail.com) and clicked Subscribe. The button changed to "Subscribing...". Became disabled. The server got the request sent the email and gave an HTTP 200 OK status code. The website then showed "Successfully subscribed! Welcome email sent.". Cleared the input field.

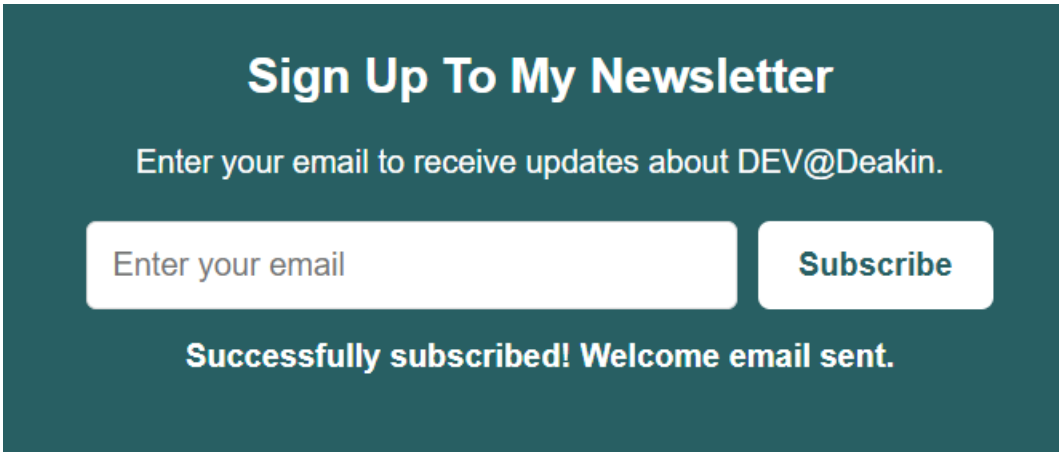
• **Email Delivery Verification:** I looked in my email inbox. Made sure that the automated welcome email from DEV@Deakin arrived successfully.

• **Terminal Status 200 Verification:** I looked at the backend terminal console after the subscription request. The console correctly logged "Welcome email sent successfully". Printed the message "Email API Status: 200".

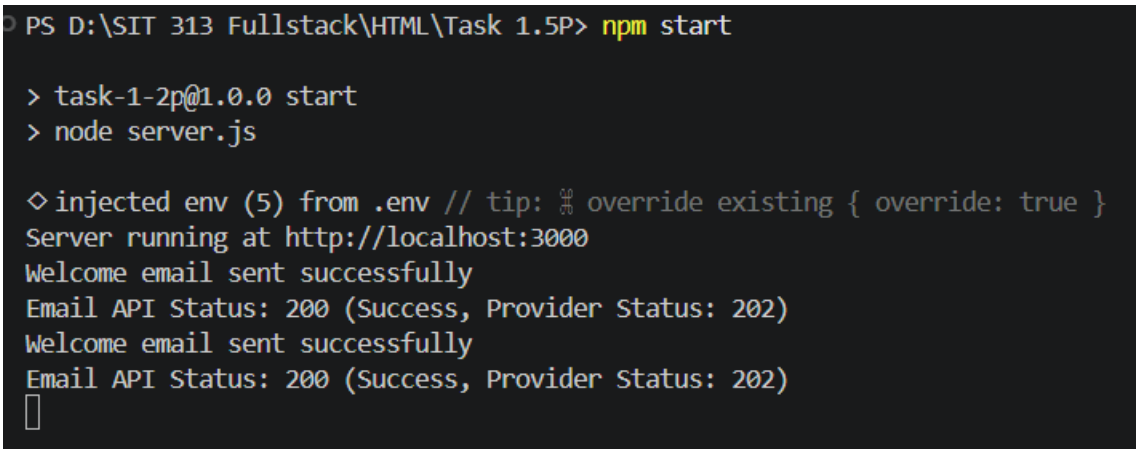
• **Backend Offline Handling:** I turned off the Express backend server. Tried to submit the form again. The try-catch block caught the connection error. Showed "Could not connect to the server. Please ensure the backend is running.", without making the application crash.

Screenshots

DEV@Deakin Web Platform



Terminal Showing Successful Status 200 Message



Submission Links

- **GitHub Repository:**

Conclusion

I successfully connected my React frontend to the Express.js backend, for the DEV@Deakin web platform. The React frontend and the Express.js backend system handles user input validation sends HTTP requests provides proper HTTP response status codes uses try-catch error handling and sends automated email notifications through SendGrid.